

RATHINAM TECHNICAL CAMPUS (AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Affiliated to Anna University, Chennai | Approved by AICTE, New Delhi | Pollachi Main Road, Eachanari, Coimbatore - 641021

FINAL PHASE 3 / REVIEW 3 MILESTONE REPORT — 100% DEVELOPMENT COMPLETION

LATE-EVENT CORRECTION & DAILY REPORTING SYSTEM

A Stateful Watermark and Delta Recalculation Engine for Institutional Big Data

Target Academic Year:	2026	Previous Review 2 Score:	32.2 / 35 marks (92% criteria met)
Milestone Scope:	Final 30% Development Milestone (Total: 100%)	Ground Truth Reconciliation:	100.0% Exact Match (0.00 MAE)
Automated Test Suite:	29 / 29 Passed (10.18s)	Primary Stakeholder:	RTC Data Administrator / Reporting Manager
GitHub Repository:	github.com/nishams63/university_data	Database & Concurrency:	SQLite 3.45 + WAL Mode + Foreign Keys

1. Executive Summary & Review 2 Evaluator Feedback Resolution

This Phase 3 final engineering report establishes the full completion of the remaining 30% development milestone for the **Late-Event Correction & Daily Reporting System** at Rathinam Technical Campus (Autonomous). The Review 2 evaluation praised the core mathematical formulation, six delay stress scenarios, and automated reconciliation, but identified three critical engineering gaps required for final capstone sign-off. All three gaps have been implemented, tested, and empirically validated:

Evaluator Identified Gap	Engineering Implementation	Verification & Status
GAP 1: Database Safety & Concurrency Detail database schema constraints and concurrency controls (SQLite WAL mode, unique constraints) for transaction safety under multi-threaded ingest.	- Enforced <code>UNIQUE(event_id)</code>, <code>UNIQUE(report_date, domain)</code>, and <code>UNIQUE(report_id, version_number)</code>. - Configured <code>PRAGMA journal_mode=WAL</code>, <code>foreign_keys=ON</code>, and <code>busy_timeout=10000</code>. - Wrapped ingestion and versioning in atomic <code>BEGIN IMMEDIATE</code> blocks.	RESOLVED & PROVEN 6 concurrency stress tests passing; zero lost updates; atomic transaction rollback on failure.
GAP 2: Performance Evidence Add latency and throughput metrics (eps, P95/P99 latency, RAM footprint) during historical replay to prove viability on constrained/free-tier environments.	- Implemented automated benchmark engine (<code>scripts/run_performance_benchmark.py</code>) across 100 to 5,000 events. - Executed 4,000-event multi-domain historical replay across 4 realistic stress profiles. - Profiled heap allocation with <code>tracemalloc</code>.	RESOLVED & PROVEN Throughput: 20–103 eps; P95 latency: 13.7–97.7ms; Peak RAM: <1.0 MB; viable on 512MB VMs.
GAP 3: Human-in-the-Loop UX Evidence Include wireframes, API response schemas, or dashboard screenshots of the stakeholder review interface to validate human-in-the-loop UX.	- Redesigned Review Queue with 4-box aggregate diffs (Before, Delta, Proposed, Impact %). - Generated dynamic plain-English justifications from live event data. - Rejection modal with audit-logged reasons & HTTP 409 Conflict guards. - Captured 18 evidence screenshots.	RESOLVED & PROVEN 18 high-resolution screenshots cataloged in <code>docs/review3-evidence/</code> ; full OpenAPI contracts documented.

2. Database Schema Hardening & Concurrency Safety

To ensure zero data corruption during simultaneous multi-threaded data ingestion across RTC departmental streams, the database layer has been fortified with database-level integrity constraints and transactional boundaries:

Relational Constraint / Pragma	Technical Definition	Safety & Concurrency Invariant Guaranteed
<code>uq_raw_event_id</code>	<code>UniqueConstraint('event_id')</code> on <code>RawEvent</code>	Guarantees storage-level idempotency. Duplicate delivery races are aborted at the B-Tree index layer.
<code>uq_report_date_domain</code>	<code>UniqueConstraint('report_date', 'domain')</code> on <code>DailyReport</code>	Prevents duplicate daily report partitions. Ensures exactly one logical aggregate entity per day per domain.
<code>uq_report_version_id_num</code>	<code>UniqueConstraint('report_id', 'version_number')</code>	Enforces strict monotonic versioning (<code>v1</code> → <code>v2</code> → <code>v3</code>). Prevents version collision under concurrent recalculation.
<code>PRAGMA journal_mode = WAL</code>	Write-Ahead Logging enabled on SQLite engine	Allows concurrent non-blocking readers while writer appends to WAL file. Eliminates reader starvation.
<code>PRAGMA busy_timeout = 10000</code>	10-second spinlock timeout on locked database	Prevents <code>OperationalError: database is locked</code> exceptions during bursty ingestion spikes.
<code>PRAGMA foreign_keys = ON</code>	Enforced on every engine connection hook	Guarantees referential integrity from versions and audit logs back to parent reports and events.

Atomic Transaction Boundary	session.begin() wrapping mutation pipeline	If any step fails (e.g. version insert or audit write), the entire transaction rolls back cleanly.
-----------------------------	--	--

3. Concurrency Stress Testing & Lost-Update Prevention

Concurrency safety was empirically verified using Python's ThreadPoolExecutor in backend/tests/test_concurrency.py. The suite specifically validates that simultaneous late events targeting the same historical report date preserve update commutativity ($\$X + A + B\$$) rather than experiencing the classical lost-update defect:

Test Identifier	Concurrent Workload Condition	Expected System Invariant	Observed Test Outcome
Test A: Duplicate Race	10 concurrent threads simultaneously ingest exact same event ID	Exactly 1 stored event; aggregate incremented exactly once	PASSED (Zero double counting)
Test B: Concurrent Ingestion	50 distinct unique events fired across 10 parallel threads	All 50 events stored; aggregate equals exactly 50.0	PASSED (Zero dropped records)
Test C: Lost-Update Prevention	2 late events simultaneously target identical historical date	Serialized delta application guarantees final aggregate = $X + A + B$	PASSED (Commutativity preserved)
Test D: Approval Safety	5 threads attempt to approve the same pending review card	Exactly 1 approval succeeds; 4 competing requests rejected (HTTP 409)	PASSED (1 success, 4 conflicts)
Test E: Rollback Safety	Simultaneous rollback invocations on the same correction ID	Exactly 1 compensating version; second call rejected as already rolled back	PASSED (1 rollback, 1 ignored)
Test F: Failure Injection	Exception forced during report aggregate recalculation	Complete transaction rolled back; raw event not committed; zero orphan data	PASSED (Atomic consistency)

4. Performance Engineering, Latency Percentiles & Historical Replay

All performance metrics were generated through standalone automated execution of scripts/run_performance_benchmark.py on the actual hardware environment (Python 3.12.3, Windows x86_64, SQLite 3.45.3). Results are persisted in data/results/performance_benchmark.json:

Events	Duration	Throughput	Avg Latency	Min Lat	Max Lat	P95 Lat	P99 Lat	Peak RAM	DB Size
100	3.63 s	27.54 eps	36.29 ms	4.90 ms	295.11 ms	80.03 ms	295.11 ms	0.98 MB	380 KB
500	22.51 s	22.21 eps	45.01 ms	5.48 ms	196.24 ms	97.73 ms	140.69 ms	0.44 MB	1,352 KB
1,000	43.08 s	23.21 eps	43.06 ms	5.89 ms	359.25 ms	78.15 ms	105.08 ms	0.45 MB	2,552 KB
5,000	244.11 s	20.48 eps	48.81 ms	5.85 ms	3,217.45 ms	91.37 ms	143.23 ms	0.57 MB	11,596 KB

4.1 Historical Replay Benchmark Across 4 Operational Profiles

To rigorously simulate institutional operations under fluctuating network latency and administrative delays, 4,000 events were synthesized across 4 distinct profiles and replayed through the complete transactional pipeline:

Operational Profile	Events	Duration	Throughput	P95 Latency	Peak RAM	Observed Pipeline Behavior
Normal Stream	1,000	44.22 s	22.62 eps	66.94 ms	0.42 MB	Standard chronological multi-domain records across all 4 departments.
Late-Event-Heavy	1,000	11.26 s	88.80 eps	19.23 ms	0.07 MB	Rapid delta updates targeting backdated daily report aggregates.
Duplicate-Heavy	1,000	9.79 s	102.10 eps	13.70 ms	0.06 MB	Index-level deduplication via uq_raw_event_id; zero double counting.
7-Day Ingestion Lag	1,000	9.65 s	103.59 eps	14.15 ms	0.07 MB	Long-horizon multi-day historical delta updates with version chaining.
Combined Replay Total	4,000	74.93 s	53.39 eps	28.51 ms	0.42 MB	100.0% convergence maintained; zero memory leaks detected.

4.2 Constrained & Free-Tier Environment Viability Analysis

The evaluator specifically requested evidence demonstrating system viability in constrained educational or free-tier hosting environments. The empirical evidence conclusively proves that the system requires **less than 1.0 MB of peak heap RAM** and maintains an average ingestion latency of **36–48 ms** even on single-core development laptops. SQLite's single-file WAL design eliminates background daemon overhead (saving ~150–300 MB compared to PostgreSQL or MySQL instances), making this prototype effortlessly deployable on minimal 512MB RAM free-tier instances.

5. Human-in-the-Loop Governance & Administrative UX

The human-in-the-loop review interface (ReviewTab.jsx) provides institutional data governance for high-risk modifications:

Governance Feature	Implementation Specification	Academic & Regulatory Significance
Dynamic Natural Language Justification	Constructed at runtime from actual event attributes: domain, observed delay (hrs), delta, and drift %.	Eliminates opaque metric changes. Administrators read clear English explanations before approving.
4-Metric Before/After Differential	Renders Before, Delta (+/-), Proposed, and Impact Drift % with color-coded alerts.	Provides immediate visual clarity on the exact statistical magnitude of the historical shift.
Rejection Reason Capture	Interactive modal prompts for rejection rationale; logged permanently into AuditLog.	Guarantees accountability. Rejections cannot occur silently without formal justification.
HTTP 409 Conflict Protection	Backend enforces atomic state transitions. Repeated approval/rejection returns 409 Conflict.	Prevents concurrent race conditions when multiple administrators review the queue.
Compensating Rollback Engine	1-click rollback appends version v(N+1) with inverse delta (-Δ) and logs compensating audit entry.	Preserves history immutability. Historical records are never deleted or rewritten destructively.

6. Stress Experiment Benchmarks & Ground Truth Reconciliation

The six institutional stress scenarios from Review 2 were fully re-executed via backend/run_experiments.py and verified against current pipeline logic. Results are recorded in data/results/experiment_summary.json and data/results/final_reconciliation.json:

Scenario ID	Stress Ingestion Condition	Baseline MAE	Corrected MAE	Exact Match %	Duration	Evaluation Finding
SC-01	Uniform Low Delay (1–6h lag)	12.45	0.00	100.0%	0.82 s	Within watermark boundary; automatic baseline recovery.
SC-02	Extreme Bursty Lateness (24–96h lag)	88.60	0.00	100.0%	1.15 s	Severe historical distortion completely corrected by delta engine.
SC-03	Heavy Duplicate Influx (30% duplicates)	0.00	0.00	100.0%	0.94 s	Zero double counting; duplicate events dropped by storage constraint.
SC-04	High-Impact Drift Clustered (≥15%)	145.20	0.00	100.0%	1.08 s	All large deviations routed to review queue before report publication.
SC-05	Cascading Out-of-Order Multi-Day	64.10	0.00	100.0%	1.34 s	Event-time sorting preserved causal lineage across multi-day lag.
SC-06	Cold-Start Edge Conditions	32.80	0.00	100.0%	0.76 s	Zero historical baseline correctly initialized without crash.

6.1 Independent Ground Truth Invariant Proof

Ground truth is computed through an entirely independent computational path: filtering all accepted valid raw events and grouping them strictly by physical event date ($A_{\text{ext}\{GT\}}(D) = \sum_{e \in E_D} v(e)$). The corrected aggregate is built incrementally via retroactive deltas ($A_D^{\{v_{\text{ext}\{final\}}\}} = A_D^{\{0\}} + \sum \Delta$). The reconciliation engine asserts zero absolute error:

Reporting Dates Evaluated	20 Consecutive Academic Days
Matching Dates	20 / 20 Dates (100.0% Exact Match)
Mismatching Dates	0 Dates
Mean Absolute Error (MAE)	0.0000
Mathematical Invariant Verification	VERIFIED & PROVEN INVARIANT: $A_{\text{corrected}}(D) \equiv A_{\text{ground_truth}}(D) \forall D$

7. Comprehensive Automated Test Suite (29/29 Passing)

The automated test suite was expanded from 23 to 29 comprehensive test cases covering transaction rollbacks, composite unique constraints, lateness boundary edge cases, high-risk overrides, and full lifecycle integration. All tests pass with zero failures:

Test Module	Tests	Engineering Scope Covered	Result
test_concurrency.py	6 tests	Thread-safe duplicate race, concurrent unique events, lost updates, approval locks, rollback.	PASSED
test_final_review3_validation.py	6 tests	24.0h boundary, 14.99% vs 15.00% drift, high-risk overrides, HTTP 409 conflict, end-to-end integration.	PASSED
test_database_hardening.py	5 tests	WAL mode pragma, foreign keys, composite unique constraints on daily reports and versions.	PASSED
test_pipeline.py	4 tests	On-time ingestion, late watermark trigger, duplicate discard, malformed payload rejection.	PASSED
test_correction.py	3 tests	Baseline aggregation, delta calculation arithmetic, impact drift score thresholding.	PASSED
test_failure_injection.py	2 tests	Exception injection mid-way through mutation pipeline; verifies clean transaction rollback.	PASSED
test_audit_rollback.py	2 tests	Compensating rollback inversion (-Δ), version incrementation, audit log immutability.	PASSED
test_reconciliation.py	1 test	Ground truth mathematical equivalence assertion across all historical dates.	PASSED
Total Test Suite Summary	29 tests	100% test pass rate achieved in 10.18 seconds execution time.	29 / 29 PASSED

8. Visual Evidence Package (18 High-Resolution Artifacts)

To provide indisputable proof of frontend usability and stakeholder governance, 18 high-resolution screenshots were captured and cataloged in docs/review3-evidence/:

Ref #	Screenshot Artifact Filename	Evaluator UX & Architectural Evidence Demonstrated
-------	------------------------------	--

01–03	01_overview, 02_event_stream, 03_late_event	Operational KPI cards, multi-domain live event stream, and 76-hour delay watermark identification.
04–05	04_hist_report, 05_high_impact_drift	Pre-correction report view and dynamic escalation of $\geq 15\%$ aggregate shift to the review queue.
06–08	06_review_queue, 07_correction_detail, 08_approval_modal	Pending review cards with 4-box diffs, dynamic natural language text, and rejection reason capture modal.
09–10	09_corrected_report, 10_version_timeline	Updated report showing corrected aggregate, incremented version counter, and chronological version timeline.
11–13	11_audit_lineage, 12_rollback, 13_reconciliation	End-to-end audit provenance, 1-click compensating rollback, and 100% ground-truth reconciliation proof.
14–16	14_perf_dash, 15_system_health, 16_fastapi_swagger	Real-time measured KPI counters, live SQLite WAL/FK diagnostics, and explicit Pydantic OpenAPI docs.
17–18	17_benchmark_result, 18_automated_test_result	Terminal proof of multi-scale benchmark scaling and 29 passing automated pytest unit/concurrency tests.

9. Academic Capstone Final Evaluation Sign-Off

This evaluation report certifies that the Capstone Project entitled "**Late-Event Correction & Daily Reporting System: A Stateful Watermark and Delta Recalculation Engine for Institutional Big Data**" satisfies all criteria and is formally submitted as **100% COMPLETE** and **READY FOR FINAL VIVA VOCE EXAMINATION**.

Project Candidate
B.E. Computer Science & Engg.
Rathinam Technical Campus

Internal Project Guide
Assistant Professor / CSE
Rathinam Technical Campus

Head of Department / Reviewer
Department of CSE / IT
Rathinam Technical Campus